Reviewer Pack — Minimal Rank of Algebraic K-Theory over Tseitin Circuit Widt…

Entry ee001464294f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 04:46:39 UTC

Minimal Rank of Algebraic K-Theory over Tseitin Circuit Width

Entry ID: `ee001464294f` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-27 04:46:39 UTC

1. Conjecture

Field A (mathematical branch): Algebraic K-theory **Field B** (complexity object): Complexity Theory: Tseitin Circuit Width

Statement:

> {'sentence_1': 'For a Tseitin formula on an n-node expander graph G, the minimal rank of the algebraic K-theory group K(G) is at least 2^(Ω(ω(G)))', 'sentence_2': 'where ω(G) is the tree-width of G.', 'sentence_3': 'Equivalently, for any Tseitin formula on an n-node graph G with tree-width ω(G), the Resolution complexity of the formula is at least 2^(Ω(ω(G)))'}

Rationale (proposer's reasoning):

> {'sentence_1': 'Algebraic K-theory provides a rich algebraic invariant that captures essential information about the connectivity and structure of graphs.', 'sentence_2': 'The conjecture posits that this invariant can be used to derive lower bounds on the Resolution complexity, which is a key resource for proving hardness results in complexity theory.', 'sentence_3': 'If true, it would provide a novel approach to studying the computational hardness of Tseitin formulas.'}

Taxonomy category: `TSEITIN_RES` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `b1fa4f3e9b806134`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For the conjecture on minimal rank of algebraic K-theory over Tseitin circuit width, support is indicated if the mean rank across all seeds is at least 2^(Ω(ω(G))) and no seed's rank exceeds this value by more than a factor of two.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `algebraic K-theory AND Tseitin circuit width` - `Resolution complexity AND tree-width IN algebraic K-theory` - `minimal rank K(G) AND Tseitin formula IN complexity theory`

Top relevant hits considered: - [http://arxiv.org/abs/0912.2255v3] Test ideals via algebras of p^{-e}-linear maps - [http://arxiv.org/abs/1812.11710v1] Geometric Satake correspondence for affine Kac-Moody Lie algebras of type A - [http://arxiv.org/abs/1705.03356v2] Growth in varieties of multioperator algebras and Groebner bases in operads - [http://arxiv.org/abs/2001.01608v2] The plethory of operations in complex topological K-theory - [http://arxiv.org/abs/1601.06285v3] Twisted iterated algebraic K-theory and topological T-duality for sphere bundles - [http://arxiv.org/abs/2103.15474v4] Hermitian K-theory via oriented Gorenstein algebras - [http://arxiv.org/abs/2604.05701v1] Measurement of the CKM angle γ in $B^{\pm} \to D(\to K^0_S h'^+ h'^-) h^{\pm}$ dec - [http://arxiv.org/abs/1411.4413v2] Observation of the rare $B^0_s \to \mu^+\mu^-$ decay from the combined analysis of CMS and LHCb data

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=0, elapsed=3.0s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def generate_expander_graph(n):
    if n <= 2:
        return None
    graph = [[] for _ in range(n)]
    for i in range(1, n):
        neighbors = random.sample(range(i), min(i-1, 3))
        for j in neighbors:
            graph[i].append(j)
            graph[j].append(i)
    return graph

def tree_width(graph):
    if not graph:
        return 0
    n = len(graph)
    visited = [False] * n
    parent = [-1] * n

    def dfs(node, depth):
        visited[node] = True
```

```
        max_depth = depth
        for neighbor in graph[node]:
            if not visited[neighbor]:
                parent[neighbor] = node
                max_depth = max(max_depth, dfs(neighbor, depth + 1))
        return max_depth

    root = 0
    while len(graph[root]) > 2:
        root += 1
    return dfs(root, 0)

def algebraic_k_theory_rank(graph):
    if not graph:
        return 0
    n = len(graph)
    k_theory_matrix = [[Fraction(0) for _ in range(n)] for _ in range(n)]

    def gaussian_elimination(matrix):
        m = len(matrix)
        n = len(matrix[0])
        rank = 0

        for i in range(m):
            if rank >= n:
                break
            pivot_row = i
            while pivot_row < m and matrix[pivot_row][rank] == Fraction(0):
                pivot_row += 1
            if pivot_row == m:
                continue

            matrix[i], matrix[pivot_row] = matrix[pivot_row], matrix[i]

            for j in range(m):
                if i != j:
                    factor = -matrix[j][rank] / matrix[i][rank]
                    for k in range(n):
                        matrix[j][k] += factor * matrix[i][k]

            rank += 1

        return rank

    for i in range(n):
        k_theory_matrix[i][i] = Fraction(1)

    return gaussian_elimination(k_theory_matrix)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    total_rank = 0
    instances_tested = 0

    for n in n_values:
```

```
        graph = generate_expander_graph(n)
        if not graph:
            continue

        rank = algebraic_k_theory_rank(graph)
        total_rank += rank
        instances_tested += 1

    mean_rank = total_rank / instances_tested if instances_tested > 0 else 0
    conjecture_holds = mean_rank >= 2 ** (math.log(n, 2) * math.log(n, 2))
    counterexample = "" if conjecture_holds else f"n={n}, rank={rank}"

    return {
        "metric_name": "algebraic_k_theory_rank",
        "metric_value": mean_rank,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else list(range(2, 30)) + list(map(lamb

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_
        results.append(result)

    mean_rank = sum(r["metric_value"] for r in results) / len(results) if results else 0
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results) if results

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std=0.0 support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"n={result['counterexample']}\", first_failing_s
    else:
        print(f"RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
cture_holds": False, "counterexample": "n=40, rank=40"}
TRIAL: {"seed": 463, "metric_name": "algebraic_k_theory_rank", "metric_value": 20.0, "instances_test
TRIAL: {"seed": 503, "metric_name": "algebraic_k_theory_rank", "metric_value": 20.0, "instances_test
TRIAL: {"seed": 547, "metric_name": "algebraic_k_theory_rank", "metric_value": 20.0, "instances_test
TRIAL: {"seed": 593, "metric_name": "algebraic_k_theory_rank", "metric_value": 20.0, "instances_test
TRIAL: {"seed": 631, "metric_name": "algebraic_k_theory_rank", "metric_value": 20.0, "instances_test
TRIAL: {"seed": 677, "metric_name": "algebraic_k_theory_rank", "metric_value": 20.0, "instances_test
TRIAL: {"seed": 727, "metric_name": "algebraic_k_theory_rank", "metric_value": 20.0, "instances_test
TRIAL: {"seed": 773, "metric_name": "algebraic_k_theory_rank", "metric_value": 20.0, "instances_test
```

```
TRIAL: {"seed": 821, "metric_name": "algebraic_k_theory_rank", "metric_value": 20.0, "instances_test
TRIAL: {"seed": 877, "metric_name": "algebraic_k_theory_rank", "metric_value": 20.0, "instances_test
TRIAL: {"seed": 929, "metric_name": "algebraic_k_theory_rank", "metric_value": 20.0, "instances_test
RESULT: FALSIFIED counterexample="n=n=40, rank=40", first_failing_seed=11
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> The test has only considered instances with n ≤ 40, which is too small to draw a definitive conclusion about the conjecture's validity. The metric does not necessarily scale trivially with n, and it is possible that for larger values of n, the conjecture might hold.

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Safety rail: critic_challenge_falsified | original: The test results indicate that for n=40, the rank of the algebraic K-theory group exceeds the conjectured value by a factor of two, thus falsifying th | next: Further investigation is needed to determine if the conjecture holds for larger values of n. It would be beneficial to test with graphs of higher node counts and analyze the behavior of the rank as n increases.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12500
2	preregistration	ollama_remote	glm4:latest	0	0	5852
3	novelty	ollama_remote	glm4:latest	0	0	4614
4	novelty	ollama_remote	glm4:latest	0	0	6657
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19107
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12690
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11257
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12644
9	critic	ollama_remote	glm4:latest	0	0	14281
10	judge	ollama_remote	glm4:latest	0	0	5895

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 105497 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/ee001464294f.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/ee001464294f.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/ee001464294f.tar.gz` (if generated)